Code Jam 2009 - Round 1B

Analysis: Decision Tree

The hard part here was parsing the tree. An easy way to do this is by using a technique called *recursive descent*. The tree grammar is defined recursively, so it makes sense to parse it recursively, too. Here is a solution in Python.

```
import re
import sys
inp = sys.stdin

tokens = None
ti = -1

def ReadInts():
  """Reads several space-separated integers on a line.
  """
  return tuple(map(int, inp.readline().strip().split(" ")))

def NextToken():
  """Consumes the next token from 'tokens'.
  """
  global ti
  assert 0 <= ti < len(tokens)
  ti += 1
  return tokens[ti - 1]

def ParseNode():
  """Parses from 'tokens' and returns a tree node.
  """
  global ti
  assert NextToken() == "("
  node = {"weight": float(NextToken())}
  tok = NextToken()
  if tok == ")":
    return node
  node["feature"] = tok
  node[True] = ParseNode()
  node[False] = ParseNode()
  assert NextToken() == ")"
  return node

def ParseTree(s):
  """Initializes 'tokens' and 'ti' and parses a tree.
  """
  global tokens
  global ti
  s = re.compile(r"\(").sub(" ( ", s)
  s = re.compile(r"\)").sub(" ) ", s)
  s = re.compile(r"[ \n]+").sub(" ", " %s " % s)
  tokens = s[1:-1].split(" ")
  ti = 0
  return ParseNode()
```

```
def Evaluate(tree, features):
  ans = tree["weight"]
  if "feature" in tree:
    ans *= Evaluate(tree[tree["feature"] in features], features)
  return ans

if __name__ == "__main__":
  N = ReadInts()[0]
  for prob in xrange(1, N + 1):
    n_lines = ReadInts()[0]
    lines = [inp.readline() for _ in xrange(n_lines)]
    tree = ParseTree(" ".join(lines))
    n_queries = ReadInts()[0]
    print "Case #%d:" % prob
    for _ in xrange(n_queries):
      features = set(inp.readline().strip().split(" ")[2:])
      print "%.7f" % Evaluate(tree, features)
```

For each test case, we read the 'n_lines' lines containing the tree definition, we glue them together using spaces and pass the resulting string to ParseTree().

In ParseTree(), we do some "massaging" to make the string easier to parse. First, we put spaces around each parenthesis by using two simple regular expressions. Next, we replace each sequence of whitespace characters by a single space and make sure there is always exactly one space character at the beginning and the end of the input. Finally, we strip off the leading and trailing spaces and split the rest into tokens.

The ParseNode() function does the rest. It uses the NextToken() function to read one token at a time from the 'tokens' list and returns a simple dictionary representation of a tree node.

Once we have the tree as a dictionary, we then use Evaluate() to do a tree traversal from the root to a leaf and compute the answer for each input animal.

Using the parsers built into dynamic languages

A number of contestants have noticed that there is an even easier way to parse the tree. Most dynamic, interpreted languages give you access to their built-in parser, and by manipulating the input a little bit, it is possible to use make the language's interpreter do the parsing for you! This means using "eval" in JavaScript or Python, or "read" in Lisp. Check out some of the shortest solutions at 1b-a.pastebin.com/d4631e678.

Code Jam 2009 - Round 1B

Analysis: Square Math

The problem would be a lot easier if there were only plus signs used. With the presence of the negative signs, a valid expression might get to a large value in the middle, then decrease back to the value we are looking for. Will that be the shortest answer for a certain query at all?

The best path cannot be long

First of all, let us denote a non-zero digit in the square *pos* if it has at least one neighbor that is a plus sign; call it *neg* if it has one minus sign as its neighbor. A digit can be both pos and neg. We assume there are both pos digits and neg digits in the square. Let **q** be the value we are looking for.
Let g be the gcd (greatest common divisor) of all the digits in the square. If g is not 1, in order to find a solution for q, it must be a multiple of g. Dividing everything by g, we may assume from now on that the gcd of all the numbers in the square is 1. The situation is further simplified when all our numbers are between 0 and 9 -- in this case, the above implies that there must be two digits, **a** and **b**, in the square, such that gcd(a, b) = 1.

Case 1. **a** is pos and **b** is neg. Take any shortest path **P** from **a** to **b** in the square. Let **q'** be the value of this path. q' is between [-200, 200]. Let t = q - q'. We prove the case t ≥ 0; the other case is similar and left to the readers.
Since gcd(a, b)=1, one of the numbers t, t+b, t+2b, ..., t+(a-1)b is a multiple of **a**. This means we can find non-negative numbers x and y such that ax - by = t, where y < a (therefore x < t/a + b). We start from **a**, since it is a positive digit, we use the plus sign to repeat at **a** x times, then follow **P**. After reaching **b**, we use the minus sign to repeat y times. The path just described evaluates to the query **q**.

Case 2. Both **a** and **b** are pos, there is a neg digit **c**. If **c** is co-prime with either **a** or **b**, we handle it as Case 1. Otherwise c has to be 6.
Pick any shortest path **P** that connects **a**, **b**, and **c**. Suppose it evaluates to q'. Pick a non-negative z such that q - q' + 6z ≥ ab - a - b + 1. We use a basic fact in number theory here

> If positive integers a and b are co-prime, then for any t ≥ ab - a - b + 1, there exist non-negative integers x and y such that ax + by = t.

To get an answer for query q, we use the path **P**, repeat **a** x times with plus sign, **b** y times with plus sign, and **c** z times with minus sign.

Case 3. Both **a** and **b** are neg. This is similar to Case 2, and we leave it to the readers.

Thus we proved that for any solvable query, there is a path that is not long, as well as any partial sum on the path cannot be too big. The rough estimate shows that any of the paths cannot take more than 1000 steps. One can get a better bound by doing more careful analysis.

The algorithm

Our solution is a BFS (breadth first search). The search space consists all the tuples (r, c, v), where (r, c) is the position of a digit, and denote A(r, c, v) to be the best path that evaluates to v,

and ends at the position (r, c). We know there are at most 200 such (r, c)'s ($20^2/2$), and at most (much less than) 20000 such v's from the bound we get.

The only difference with a standard BFS is that, because of the lexicographical requirement, we may need to update the answer on a node. But we never need to re-push it into the queue, since it is not yet expanded.

Code Jam 2009 - Round 1B

Analysis: The Next Number

Let **x** be our input. We want to find the next number **y**. We denote L(*s*) to be the length of *s*, i.e., the number of digits in *s*.

Case 1. If all the digits in *x* are non-increasing, for example *x* = 776432100, then *x* is already the biggest one among the numbers in the list with L(*x*) digits. The next number, *y*, must have one more digit, that is, one more 0. Actually *y* must be the smallest one with L(*x*)+1 digits. To get *y*, we put the smallest non-zero digit in front, and all the other digits are put in non-decreasing order.

Case 2. Otherwise, *x* can be written as the concatenation *x* = *ab*, where *b* is the longest non-increasing suffix of *x*, so *d*, the last digit of *a* is smaller than the first digit of *b*. Let *d'* be the smallest among all the digits in *b* that are bigger than *d*.
Because *b* is non-increasing, *x* is the biggest number among those who has L(x) digits and starts with prefix *a*. The first L(*a*) digits of *y* must be bigger than *a*. The smallest we can do is to replace *d* with *d'*, and then for the rest digits, we arrange them in non-decreasing order.
Let us do another example. *x* = 134266530. Then *a* = 1342, *b* = 66530, *d* = 2, and *d'* = 3. The next number is *y* = 134302566.

In fact, we can unify the two cases above. Since the number of 0's is not restricted, we can just imagine there is one more 0 in the beginning of *x*, thus Case 1 is reduced to Case 2.

The above described is actually exactly the procedure to get the next permutation of a finite sequence in certain languages. Below is a solution that is essentially one line in C++. From the author:

```
deque<char> f;
...
f.push_front('0');
next_permutation(f.begin(), f.end());
if (f.front() == '0') f.pop_front();
```